

AI-Powered Photo Management System

Project Description

Photo Management Agentic AI is a comprehensive, AI-powered photo management system designed to bring intelligence and automation to how users organize, search, and share their memories. Unlike traditional photo galleries that rely on manual sorting, this system uses deep learning-based facial recognition, event classification, semantic search, and natural language understanding to create an intuitive and efficient photo experience. Whether identifying people in images, organizing them into smart folders, detecting events and locations, or delivering them across platforms, the system ensures seamless photo handling with minimal human effort.

At its core, the system integrates advanced computer vision models — SCRFD for face detection and ArcFace (DeepFace) for face embedding generation — to achieve highly accurate face recognition. Once photos are uploaded, the system automatically detects and labels familiar faces, clusters unknown faces using DBSCAN, and enables quick retrieval. Photos are also stored securely via Cloudinary with metadata tracked in an SQLite database.

Beyond recognition, the system incorporates a conversational Agentic AI Assistant powered by Google Gemini for natural language understanding, multimodal analysis, and task execution. Users can ask *"Show me photos of Priya"*, *"Find birthday photos"*, or *"Send John's pictures via email"* and the system interprets and executes the task using a multi-agent workflow. This extends functionality beyond search into smart actions like automated sharing, batch delivery, and history tracking.

Built with a modern architecture, the system uses Flask for its backend API, serving a static single-page frontend built in HTML5/CSS3/JavaScript. It integrates with Brevo SMTP for email delivery and Twilio for WhatsApp delivery. The system also incorporates Hugging Face face models for high-quality embedding generation under higher rate limits, CLIP for semantic image search, and FAISS for fast vector similarity matching.

Scenario 1: Family Photo Organization & Memory Management

Riya, a working professional, has over 15,000 photos across her phone, Google Drive, and an old laptop. After setting up Photo Management Agentic AI, she uploads her entire photo library through the web interface. The system automatically scans every image, detects faces using SCRFD, generates ArcFace embeddings, and clusters similar faces using DBSCAN. Familiar family members labeled once — 'Mom', 'Dad', 'Ananya', 'Grandma' — are recognized and tagged automatically across all photos.

New faces are flagged for labeling, allowing Riya to tag relatives or friends once. The system then organizes all photos into person-specific and event-based folders. When Riya types 'Show me

photos of Grandma from Diwali', the AI Agent uses Gemini to interpret the query, searches the photo database, and retrieves exactly the matching photos. She can further say 'Email Mom her birthday photos', and the agent composes and sends the email via Brevo SMTP — all conversationally

Scenario 2: Event Photographer Workflow Automation

Aarav, a wedding and event photographer, deals with thousands of photos after every event. After uploading event photos to Photo Management Agentic AI, SCRFD detects all faces and ArcFace generates embeddings. DBSCAN groups recurring individuals — bride, groom, family members — automatically. The AI classifies each photo into event categories: 'Wedding', 'Reception', 'Candid' using Gemini multimodal analysis.

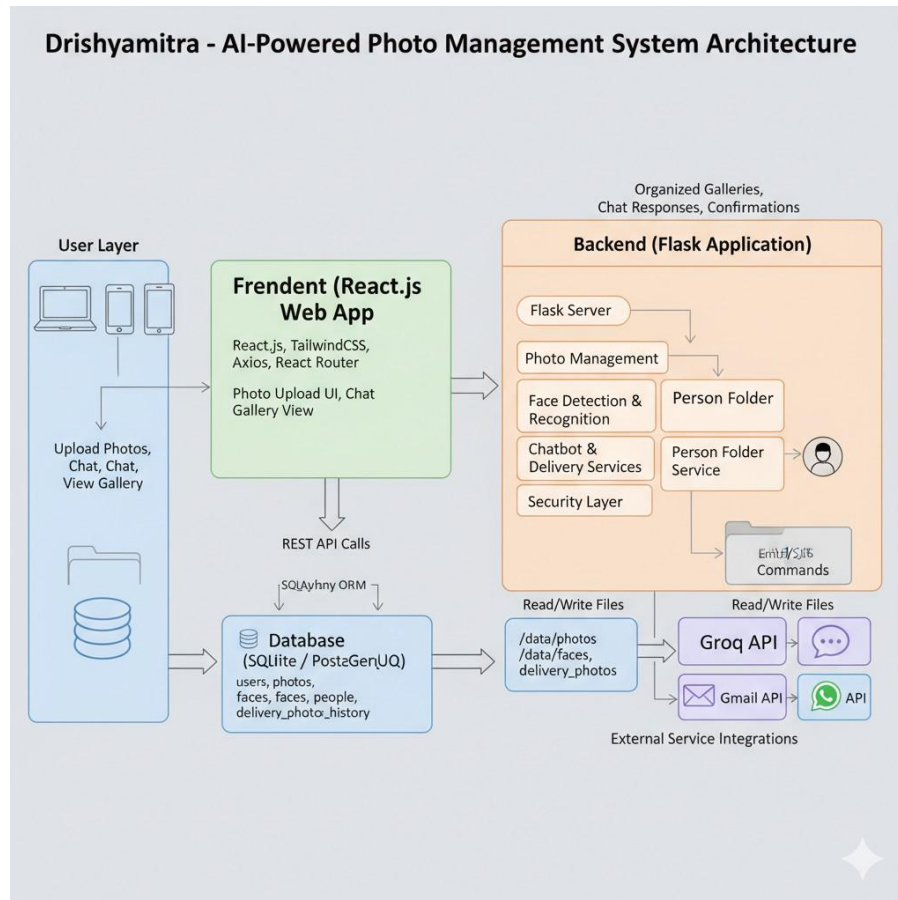
Through the built-in Agentic Chat interface, Aarav can command: 'Send family group photos to client@example.com'. The system fetches and delivers them instantly via Brevo email with delivery logs stored in the DeliveryHistory table. This saves hours of manual sorting and allows Aarav to focus on creative work.

Scenario 3: Corporate Media Management & Collaboration

At a digital marketing agency, the creative team handles thousands of images for different campaigns. The company deploys Photo Management Agentic AI as its central AI-powered media management platform. Whenever team members upload campaign photos, the system automatically detects people, identifies employees and clients via DBSCAN-clustered face embeddings, and classifies photos by event type and occasion using Gemini.

The marketing lead Neha asks the chatbot: 'Find all photos of Rohan and Priya from the Client A launch event'. The Agentic AI parses intent using Gemini, queries the database by person name and event, and instantly retrieves matches. She follows up with 'Send them via WhatsApp to our PR team', and the agent dispatches photos through Twilio WhatsApp API with automated delivery logging.

Architecture:



Pre-requisites:

Environment Requirements

- Python 3.8+ — Core runtime for Flask and all AI libraries
- pip — Python package manager for installing dependencies
- Git — For cloning the repository
- Docker (optional) — For containerized deployment

API Keys & External Services Cloudinary (Image Storage)

- Purpose: Secure cloud storage for uploaded photos with CDN delivery and transformations
- Get credentials: <https://console.cloudinary.com>
- Required keys: CLOUDINARY_CLOUD_NAME, CLOUDINARY_API_KEY, CLOUDINARY_API_SECRET

Google Gemini (Multimodal AI)

- Purpose: Event classification, celebrity recognition, and Agentic AI chat responses
- Get credentials: <https://aistudio.google.com/app/apikey>
- Required key: GEMINI_API_KEY

Hugging Face (Face Models)

- Purpose: Download SCRFD ONNX model and access higher rate-limit models
- Get token: <https://huggingface.co/settings/tokens> (Read-only permission)
- Required key: HF_TOKEN

Brevo SMTP (Email Delivery)

- Purpose: Send photos via email using Brevo transactional SMTP
- Get credentials: <https://app.brevo.com> — Settings → SMTP & API
- Required keys: SMTP_SERVER=smtp-relay.brevo.com, SMTP_PORT=587, SMTP_USERNAME, SMTP_PASSWORD, SMTP_FROM_EMAIL

Twilio WhatsApp (WhatsApp Delivery)

- Purpose: Send photos directly to WhatsApp contacts via Twilio's WhatsApp Business API
- Get credentials: <https://console.twilio.com> — WhatsApp Sandbox or Business account
- Required keys: TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN, TWILIO_WHATSAPP_FROM (e.g., whatsapp:+14155238886)

DeepFace & SCRFD Models

- SCRFD ONNX Model: Auto-downloaded from Hugging Face to assets/models/scrfd_2.5g_bnkp.onnx on first run
- ArcFace Model: Pre-loaded via DeepFace at startup — DeepFace.build_model('ArcFace')
- CLIP Model: Downloaded on first use via Hugging Face Transformers
- No manual download required — the application handles model setup automatically

Project WorkFlow

Milestone 1: Environment Setup and Project Initialization

Activity 1.1: Clone Repository and Create Virtual Environment

The project repository was cloned from GitHub to obtain the source code and project files. A Python virtual environment was then created and activated to isolate project dependencies from the system environment. This setup ensures proper dependency management, avoids version conflicts, and provides a consistent development environment for the project.

Clone the repository from GitHub and set up an isolated Python virtual environment to manage dependencies cleanly:

```
git clone https://github.com/ramavathshivaram/photo-management-agentic-ai.git
cd photo-management-agentic-ai

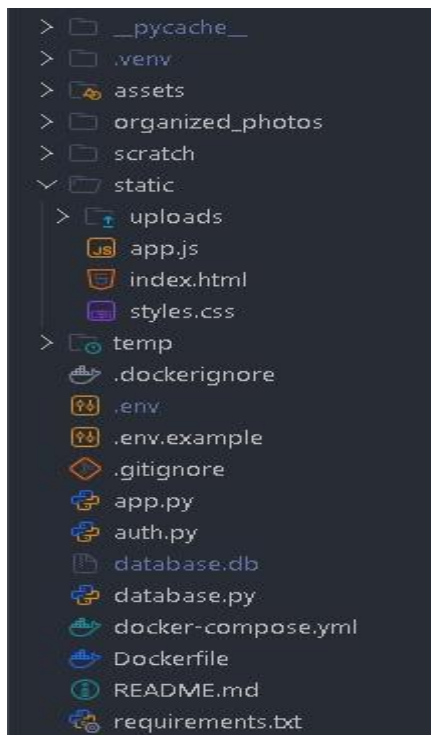
# Create virtual environment
python -m venv .venv

# Activate on Windows
.venv\Scripts\activate

# Activate on macOS/Linux
source .venv/bin/activate
```

Activity 1.2: Project Folder Structure

The project follows a flat single-service structure with the following layout:



Activity 1.3: Configure .env File:

Copy the example environment file and fill in your credentials:

```
cp .env.example .env
```

Edit the `.env` file with your actual values. All required variables are:

```
# Clouddinary
CLOUDINARY_CLOUD_NAME=
CLOUDINARY_API_KEY=
CLOUDINARY_API_SECRET=

# Authentication
JWT_SECRET=

# Email (Brevo)
SMTP_SERVER=smtp-relay.brevo.com
SMTP_PORT=587
SMTP_USERNAME=
SMTP_PASSWORD=
SMTP_FROM_EMAIL=

# Twilio WhatsApp
TWILIO_ACCOUNT_SID=
TWILIO_AUTH_TOKEN=
TWILIO_WHATSAPP_FROM=

# Google Gemini
GEMINI_API_KEY=

# Hugging Face
HF_TOKEN=
```

This separation enables the frontend and backend to be developed, tested, and deployed independently. It also improves maintainability, supports collaborative development, and simplifies future updates and deployment processes.

Activity 1.4: Install Dependencies

Install all required Python packages from the requirements.txt file:

```
pip install -r requirements.txt
```

The `requirements.txt` includes:

All required Python packages were installed using the requirements.txt file. These

dependencies provide support for web development, image processing, face recognition, machine learning, semantic search, authentication, email services, and external API integrations. Installing these packages ensures that all components of the Photo Management Agentic AI system function correctly within the configured environment.

Activity 1.5: Initialize Database and Verify Setup

The database is automatically initialized on first run. To pre-initialize manually:

```
python database.py
```

This creates `database.db` with all 5 tables (users, photos, faces, people, delivery_history) and inserts a default admin user (username: `admin`, password: `admin123`) plus sample contacts (Mom, Dad, Rahul, Priya)

Milestone 2: Core AI Pipeline — Face Detection and Recognition

Activity 2.1: SCRFD Face Detection Pipeline

When a photo is uploaded, it passes through the face detection pipeline in

`detect_and_represent()`:

- **Step 1 — Model Initialization:** SCRFD model is loaded from `assets/models/scrfd_2.5g_bnkps.onnx` at startup. If not present, it's auto-downloaded from Hugging Face.
- **Step 2 — Face Detection:** SCRFD detects faces in the image returning bounding boxes, 5-point facial keypoints (eyes, nose, mouth corners), and confidence scores.
- **Step 3 — Quality Filtering:** Faces with confidence below 0.55 are rejected.
- **Step 4 — Alignment:** 5-point keypoints are used to compute an affine transformation matrix via `cv2.estimateAffinePartial2D()`, aligning the face to a canonical 112×112 orientation for better embedding accuracy.
- **Step 5 — Embedding:** Each aligned face crop is passed to `DeepFace.represent()` using the ArcFace model to generate a 512-dimensional embedding vector.

```
# SCRFD Detector Configuration and Download
SCRFD_MODEL_PATH = os.path.join("assets", "models", "scrfd_2.5g_bnkps.onnx")
SCRFD_MODEL_URL = "https://huggingface.co/RuteML/SCRFD-face-detection-ONNX/resolve/main/2.5g_bnkps.onnx"
```

Windsurf: Refactor | Explain | Generate Docstring

```
def ensure_scrfd_model():
    if not os.path.exists(SCRFD_MODEL_PATH):
        print(f"SCRFD model not found. Downloading from {SCRFD_MODEL_URL}...")
        os.makedirs(os.path.dirname(SCRFD_MODEL_PATH), exist_ok=True)
        try:
            urllib.request.urlretrieve(SCRFD_MODEL_URL, SCRFD_MODEL_PATH)
            print(f"SCRFD model downloaded to {SCRFD_MODEL_PATH}")
        except Exception as e:
            print(f"Failed to download SCRFD model: {e}")
            raise e
```

```
# Initialize detector
scrfd_detector = None
try:
    ensure_scrfd_model()
    print("Initializing SCRFD model...")
    scrfd_detector = scrfd.SCRFD.from_path(SCRFD_MODEL_PATH)
    print("SCRFD model initialized successfully.")
except Exception as scrfd_err:
    print(f"SCRFD model initialization failed: {scrfd_err}")
```


Activity 2.2: Face Matching with FAISS

After generating embeddings, the system uses FAISS for fast approximate nearest-neighbour search:

```
def match_face_faiss(emb, known_profiles, threshold=COSINE_SIM_THRESHOLD):
    if not known_profiles:
        return "Unknown"
    try:
        embeddings = [normalize_embedding(p["embedding"]) for p in known_profiles]
        embeddings_matrix = np.vstack(embeddings).astype(np.float32)

        # ArcFace output vectors are 512 dimensions
        index = faiss.IndexFlatIP(512)
        index.add(embeddings_matrix)

        query_vector = normalize_embedding(emb).reshape(1, -1)
        distances, indices = index.search(query_vector, 1)

        best_similarity = distances[0][0]
        best_index = indices[0][0]

        # For ArcFace, inner product  $\geq 0.32$  matches
        if best_index  $\neq$  -1 and best_similarity  $\geq$  threshold:
            return known_profiles[best_index]["label"]
    except Exception as e:
        print(f"FAISS search match failed: {e}")
    return "Unknown"
```

- **All stored embeddings** are loaded from the `faces` table and normalized to unit vectors
- **FAISS IndexFlatIP** (inner product / cosine similarity) is built on the fly for the user's embeddings
- **Cosine similarity threshold**: configurable (default ~ 0.68). Faces scoring above threshold are matched to the stored person label

Unknown faces are stored with label 'Unknown' for later clustering or manual labeling

Activity 2.3: DBSCAN Clustering for Unknown Faces

Unlabeled faces are grouped automatically using DBSCAN clustering:

- All 'Unknown' face embeddings are loaded and cosine distances computed
- DBSCAN is run with `metric='cosine'` to find clusters of similar faces
- Each cluster is assigned a system-generated name ('Person_1', 'Person_2', etc.)
- Users can then rename clusters via the People Management page or merge/split clusters

Label propagation: once a face is labeled, `propagate_labels()` automatically labels similar faces across all photos

```
@app.route("/api/people/clusters", methods=["GET"])
@login_required
def get_people_clusters():
    conn = get_db()
    clusters = []
    try:
        cursor = conn.cursor()
        user_id = request.user["user_id"]

        # Fetch all faces and calculate average confidence
        cursor.execute("""
            SELECT f.id, f.label, f.embedding, f.photo_id
            FROM faces f
            JOIN photos p ON f.photo_id = p.id
            WHERE p.user_id = ?
            """, (user_id,))
        db_faces = [dict(row) for row in cursor.fetchall()]
        calculate_face_confidences(db_faces)

        by_label_conf = {}
        for f in db_faces:
            lbl = f["label"]
            if lbl != "Unknown":
                if lbl not in by_label_conf:
                    by_label_conf[lbl] = []
                by_label_conf[lbl].append(f["confidence"])

        # Query unique labeled people and select a sample face image box for preview crop
        cursor.execute("""
            SELECT f.label, p.secure_url, f.x, f.y, f.w, f.h, COUNT(DISTINCT f.photo_id) as photo_count
            FROM faces f
            JOIN photos p ON f.photo_id = p.id
            WHERE f.label != 'Unknown' AND p.user_id = ?
            GROUP BY f.label
            ORDER BY photo_count DESC
            """, (user_id,))
        rows = cursor.fetchall()
        for r in rows:
            lbl = r["label"]
            confs = by_label_conf.get(lbl, [1.0])
            avg_conf = float(np.mean(confs))

            clusters.append({
                "label": lbl,
                "secure_url": r["secure_url"],
                "x": r["x"],
                "y": r["y"],
                "w": r["w"],
                "h": r["h"],
                "photo_count": r["photo_count"],
                "avg_confidence": round(avg_conf, 3)
            })
    finally:
        conn.close()
    return jsonify(clusters)
```

Activity 2.4: Event Classification with Gemini Multimodal AI

Every uploaded photo is also classified by event type using

```
classify_photo_event_with_gemini():
```

- Image is sent to Google Gemini as a base64-encoded multimodal prompt
- Gemini returns event category (Wedding, Birthday, Festival, Travel, Sports, etc.), confidence score, and occasion type (Occasion / Non_Occasion)
- Results are stored in `photos.event`, `photos.event_confidence`, and `photos.event_occasion`
- If Gemini is unavailable, CLIP zero-shot classification is used as fallback

Milestone 3: Agentic AI Chat Assistant

Activity 3.1: Multi-Agent Architecture

The Agentic AI system at `/api/agent` implements a stateful multi-agent workflow with four specialized sub-agents:

Activity 3.2: Stateful Conversation Sessions

Agent sessions are maintained in-memory via `AGENT_SESSIONS` dictionary, keyed by `user_id`. Each session tracks:

- `state`: IDLE → CONFIRM_DELIVERY → waiting for method confirmation
- `photos`: list of photo objects retrieved by a search query
- `method`: selected delivery method (email or whatsapp)
- `person`: identified person from the query
- `recipient`: resolved recipient contact from people table

A typical agentic conversation flow:

```
User: "Show me photos of Priya"
```

```
Agent: Intent Agent → Search Agent queries DB for faces labeled "Priya"  
Returns thumbnail grid of matching photos
```

```
User: "Send them to her via WhatsApp"
```

```
Agent: Intent Agent → Delivery Agent  
Looks up Priya's WhatsApp number in people table  
Sends photos via Twilio API  
Logs delivery to delivery history table  
Returns "Sent 5 photos to Priya via WhatsApp"
```

Activity 3.3: Semantic Search with CLIP

For natural language photo queries that go beyond person names, the system uses CLIP embeddings:

- On upload, each photo's CLIP embedding is computed and stored
- On search query, the text query is encoded with CLIP's text encoder
- Cosine similarity between text embedding and all photo embeddings identifies relevant photos
- Example queries: *"beach photos"*, *"family dinner"*, *"birthday cake"*

Activity 3.4: Celebrity Recognition with Gemini

For celebrity face identification, the system uses Gemini's multimodal vision capability:

- Face crop is extracted from the detected bounding box
- Cropped face image is encoded in base64 and sent to Gemini with a recognition prompt
- Gemini returns celebrity name with confidence, or 'Unknown' if not recognized

Results are used to auto-label faces and update the people table

Milestone 4:

Activity 4.1: Email Delivery via Brevo SMTP

The `/api/delivery/email` endpoint handles email-based photo delivery:

```
@app.route("/api/delivery/email", methods=["POST"])
@login_required
def send_email():
    data = request.get_json() or {}
    recipient = data.get("recipient", "").strip()
    photo_ids = data.get("photo_ids", [])

    if not recipient or not photo_ids:
        return jsonify({"error": "Missing recipient or photo selection"}), 400

    # Verify photos ownership
    conn = get_db()
    try:
        cursor = conn.cursor()
        placeholders = ",".join(["?" * len(photo_ids)])
        cursor.execute(f"SELECT COUNT(*) FROM photos WHERE user_id = ? AND id IN ({placeholders})", [request.user["user_id"]] + photo_ids)
        if cursor.fetchone()[0] != len(photo_ids):
            return jsonify({"error": "Unauthorized photo selection"}), 403
    finally:
        conn.close()

    status, details = dispatch_delivery("email", recipient, photo_ids)

    conn = get_db()
    try:
        cursor = conn.cursor()
        timestamp = time.strftime("%Y-%m-%d %H:%M:%S")
        cursor.execute("""
            INSERT INTO delivery_history (user_id, timestamp, delivery_method, recipient, photo_ids, status, details)
            VALUES (?, ?, 'email', ?, ?, ?, ?)
            """, (request.user["user_id"], timestamp, recipient, json.dumps(photo_ids), status, details))
        conn.commit()
    finally:
        conn.close()

    if status == "failed":
        return jsonify({"error": details}), 500
    return jsonify({"message": "Email request processed", "details": details})
```

- Accepts `photo_ids` (list) and `recipient_email` in request body
- Photos are fetched from Cloudinary using their `secure_url` references
- Email is composed with photo attachments and sent via `smtplib` using Brevo SMTP credentials
- Delivery record is inserted into `delivery_history` table with timestamp, status, and details

Activity 4.2: WhatsApp Delivery via Twilio

The `/api/delivery/whatsapp` endpoint handles WhatsApp-based photo delivery:

```
@app.route("/api/delivery/whatsapp", methods=["POST"])
@login_required
def send_whatsapp():
    data = request.get_json() or {}
    recipient = data.get("recipient", "").strip()
    photo_ids = data.get("photo_ids", [])

    if not recipient or not photo_ids:
        return jsonify({"error": "Missing recipient or photo selection"}), 400

    # Verify photos ownership
    conn = get_db()
    try:
        cursor = conn.cursor()
        placeholders = ",".join(["?"] * len(photo_ids))
        cursor.execute(f"SELECT COUNT(*) FROM photos WHERE user_id = ? AND id IN ({placeholders})", [request.user["user_id"]] + photo_ids)
        if cursor.fetchone()[0] != len(photo_ids):
            return jsonify({"error": "Unauthorized photo selection"}), 403
    finally:
        conn.close()

    status, details = dispatch_delivery("whatsapp", recipient, photo_ids)

    conn = get_db()
    try:
        cursor = conn.cursor()
        timestamp = time.strftime("%Y-%m-%d %H:%M:%S")
        cursor.execute("""
            INSERT INTO delivery_history (user_id, timestamp, delivery_method, recipient, photo_ids, status, details)
            VALUES (?, ?, 'whatsapp', ?, ?, ?, ?)
        """, (request.user["user_id"], timestamp, recipient, json.dumps(photo_ids), status, details))
        conn.commit()
    finally:
        conn.close()

    if status == "failed":
        return jsonify({"error": details}), 500
    return jsonify({"message": "WhatsApp request processed", "details": details})
```

- Accepts `photo_ids` and `recipient_number` in request body
- Uses Twilio's `messages.create()` API with `media_url` pointing to Cloudinary URLs
- WhatsApp sandbox or business account sends the media message to the recipient
- Delivery status, timestamp, and recipient are logged to `delivery_history`

Activity 4.3: Delivery History Tracking

The `/api/delivery/history` endpoint returns a paginated log of all deliveries:

```
@app.route("/api/delivery/history", methods=["GET"])
@login_required
def get_delivery_history():
    conn = get_db()
    try:
        cursor = conn.cursor()
        cursor.execute("""
            SELECT id, timestamp, delivery_method, recipient, photo_ids, status, details
            FROM delivery_history
            WHERE user_id = ?
            ORDER BY id DESC
        """, (request.user["user_id"],))
        history = [dict(h) for h in cursor.fetchall()]
        return jsonify(history)
    finally:
        conn.close()
```

- All deliveries (email and WhatsApp) are stored with: timestamp, method, recipient, photo count, status, and details
- Frontend displays history in a sortable table on the Delivery History page

Failed deliveries are logged with error details for debugging

Milestone 5: Frontend -- SinglePageApplication:

Activity 5.1: Application Pages

The frontend is implemented as a Single Page Application (SPA) containing multiple modules for different functionalities. It includes pages for user authentication, dashboard analytics, photo gallery management, people and contact management, AI-powered chat interactions, and delivery history tracking. This design provides a seamless and user-friendly experience by integrating all major features within a single interface

Activity 5.2: Photo Upload Flow

The upload interface supports multi-photo upload with real-time feedback:

- User selects one or more photos via the file picker
- Files are sent as `multipart/form-data` to `POST /api/upload`
- Backend runs SCRFD detection, ArcFace embedding, Cloudinary upload, and Gemini event classification in sequence
- Response returns photo metadata including detected faces (with bounding boxes) and recognized person names

Gallery refreshes automatically to display newly uploaded photos

Activity 5.3: People & Cluster Management

The People Management page exposes full cluster management capabilities:

- View all detected people with face count and sample thumbnails
- Run DBSCAN re-clustering via `POST /api/people/cluster-dbscan`
- Rename a person cluster (updates all matching face labels across photos)
- Merge two clusters via `POST /api/people/merge-clusters` (e.g., when the same person has two separate clusters)
- Split a cluster via `POST /api/people/split-cluster` (e.g., when two people were incorrectly grouped)

Milestone 6: Testing and Deployment

Activity 6.1: Running the Application

Start the application with:

```
python app.py
```

The backend runs at <http://localhost:5000>. The single-page frontend is served at the same URL — navigate to <http://localhost:5000> in your browser.

Activity 6.2: Docker Deployment

For containerized deployment:

```
# Build the Docker image
docker build -t photo-management-agent-ai .

# Run the container
docker run -d \
  --name photo-management-agent-ai \
  -p 5000:5000 \
  --env-file .env \
  photo-management-agent-ai
```

Activity 6.3: Test Scripts:

The [scratch/](#) directory contains test and utility scripts for verification:

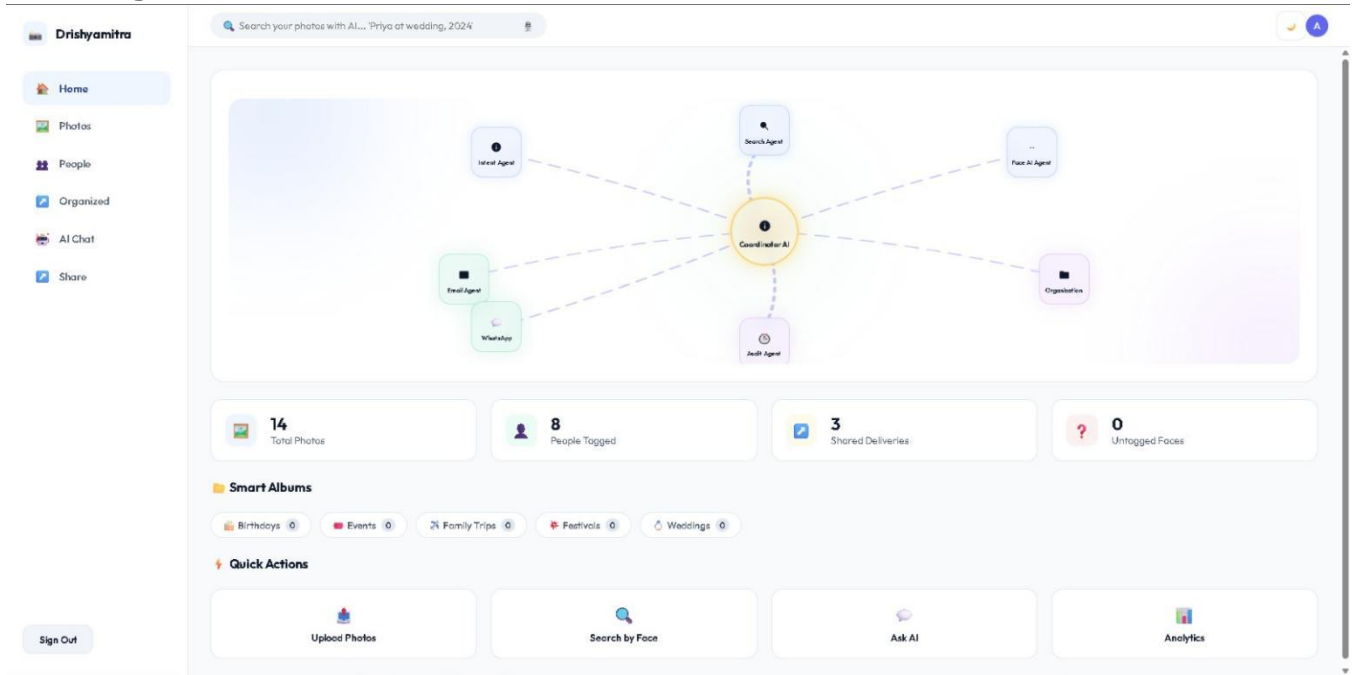
Script	Purpose
test_registration.py	Test user registration and login flow
test_chat_search.py	Test semantic chat search via /api/agent
test_chat_delivery.py	Test delivery commands through the agent
test_brevo_delivery.py	Test direct email delivery via Brevo
test_twilio_whatsapp_delivery.py	Test direct WhatsApp delivery via Twilio
test_dbscan_clustering.py	Test DBSCAN clustering on face embeddings
test_multitenant_isolation.py	Test that users cannot see each other's data
test_event_organization.py	Test photo organization by event and person
test_family_and_delivery.py	End-to-end test: upload → recognize → deliver
check_db.py	Inspect database contents for debugging
reset_db.py	Reset the database to a clean state

Activity 6.4: Authentication and Security

Security is handled through a custom JWT implementation in `auth.py`:

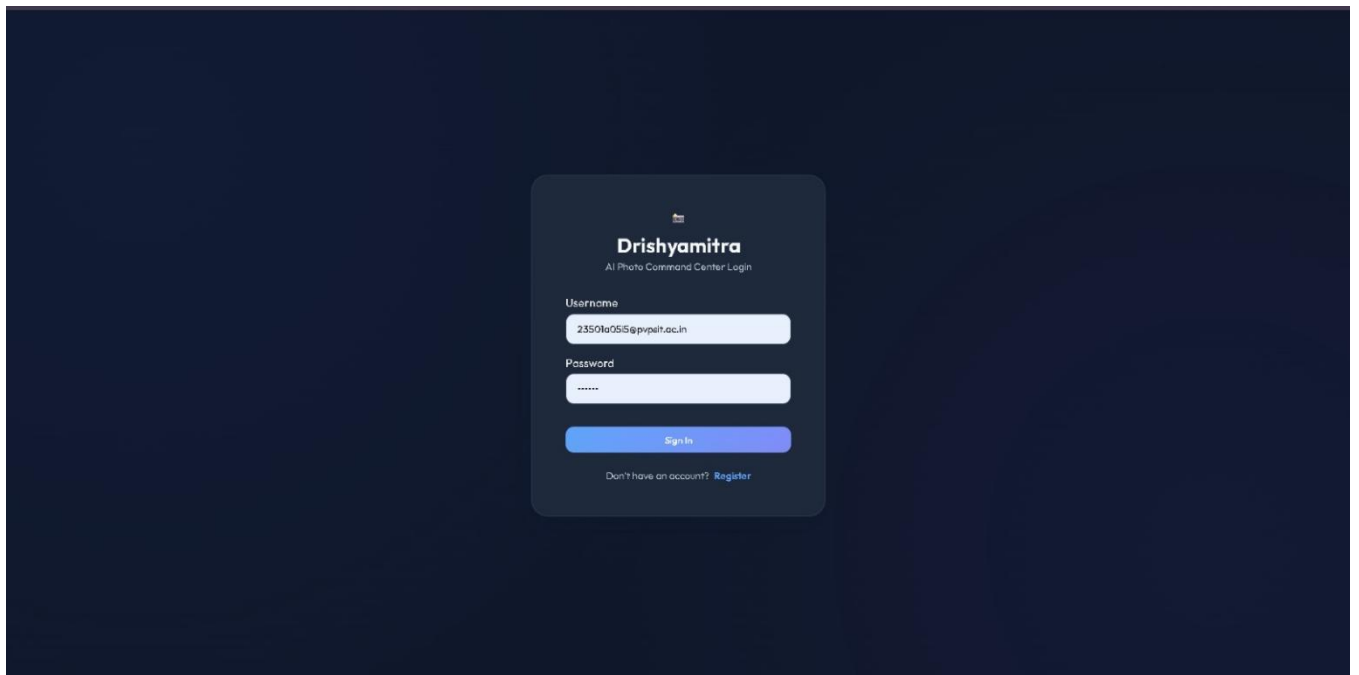
- **Token Generation:** HMAC-SHA256 signed tokens with 24-hour expiry, containing `user_id` and `username`
- **Password Hashing:** Werkzeug's `generate_password_hash()` (PBKDF2-HMAC-SHA256) for all stored passwords
- **Protected Routes:** All API routes except `/api/auth/login` and `/api/auth/register` require a valid `Authorization: Bearer <token>` header
- **Multi-tenant Isolation:** Every database query is filtered by `user_id` from the token payload, ensuring complete data isolation between users
- **Secrets Management:** All credentials stored in `.env` file, excluded from Git via `.gitignore`

Home Page



The Home Page interface for Drishyamitra. It features a sidebar on the left with navigation links: Home, Photos, People, Organized, AI Chat, and Share. The main content area includes a search bar at the top with the text "Search your photos with AI... Priya at wedding, 2024". Below the search bar is a central diagram showing a "Coordinator AI" hub connected to various agents: Travel Agent, Search Agent, Face AI Agent, Organization, Health Agent, WhatsApp, and Email Agent. At the bottom of the main area, there are four summary cards: "14 Total Photos", "8 People Tagged", "3 Shared Deliveries", and "0 Untagged Faces". Below these are "Smart Albums" (Birthdays, Events, Family Trips, Festivals, Weddings) and "Quick Actions" (Upload Photos, Search by Face, Ask AI, Analytics). A "Sign Out" button is located in the bottom left corner.

Login and SignUP page:



The Login and SignUP page for Drishyamitra. It features a dark blue background with a central login form. The form is titled "Drishyamitra" and "AI Photo Command Center Login". It includes fields for "Username" (with the example "2350ta05i5@pyelit.ac.in") and "Password" (with a masked input "*****"). A "Sign In" button is located below the password field. At the bottom of the form, there is a link "Don't have an account? Register".

Drishyamitra

Home

Photos

People

Organized

AI Chat

Share

Search your photos with AI... Priya at wedding, 2024

Upload Photos to Cloudinary

Drag and drop multiple photos here, or click to browse

Supports JPG, PNG

Photos Gallery

Filter by Name (e.g. Priya)

dd-mm-yyyy

Search by Face

Clear

3.jpeg

Uploaded 2024-06-10

Person_3 (87%)

2.jpeg

Uploaded 2024-06-10

Person_3 (73%)

1.jpeg

Uploaded 2024-06-10

Person_3 (88%) Person_1 (75%)

5.jpeg

Uploaded 2024-06-10

Person_1 (92%) Person_3 (95%) Person_2 (90%) Person_4 (90%) Person_5 (90%)

WhatsApp Image 2026-06-10 at 10...

Uploaded 2024-06-10

Person_1 (90%) Person_2 (97%)

Drishyamitra

Home

Photos

People

Organized

AI Chat

Share

Sign Out

Search your photos with AI... Priya at wedding, 2024

▼ Events 4

▶ Casual 2

▶ Family Gathering 1

▼ Family Photo 2

2.jpeg WhatsApp Image ...

▼ Travel 2

2.jpeg 3.jpeg

Chatbot Page

Drishyamitra

Home

Photos

People

Organized

AI Chat

Share

Sign Out

Search your photos with AI... Priya at wedding, 2024

Pixel Mind Assistant

Search, organize and share your memories

Clear Chat

Please enter the recipient's email address:

ramavathshiva6300@gmail.com

You are about to send **14** photo(s) to **ramavathshiva6300@gmail.com**.

Proceed? (Yes / No)

yes

Success! 14 photos have been delivered to **ramavathshiva6300@gmail.com** via Email.

System Execution Logs

> Coordinator Agent: Transaction approved by user. Dispatching...

> Audit Agent: Transaction saved in delivery logs. Status: success

Type a command...

Share:

Drishyamitra

Home

Photos

People

Organized

AI Chat

Share

Sign Out

Search your photos with AI... Priya at wedding, 2024

Dispatch Delivery Logs

Timestamp	Method	Recipient	Photos Sent	Status	Delivery Details
2026-06-30 23:55:33	EMAIL	ramavathshiva6300@gmail.com	14 photo(s)	SUCCESS	Email sent successfully to ramavathshiva6300@gmail.com with 14 attachment(s).
2026-06-30 23:45:57	WHATSAPP	+916300252005	3 photo(s)	SUCCESS	WhatsApp message sent successfully to +916300252005 via Twilio. Sent 3 media attachment(s) directly.
2026-06-30 23:45:36	EMAIL	ramavathshiva6300@gmail.com	3 photo(s)	SUCCESS	Email sent successfully to ramavathshiva6300@gmail.com with 3 attachment(s).
2026-06-30 23:44:04	EMAIL	ramavathshiva6300@gmail.com	3 photo(s)	FAILED	Email delivery failed: (526, 5/57) Unauthorized IP address

Conclusion:

Photo Management Agentic AI represents a significant advancement in intelligent, automated photo management. By combining state-of-the-art computer vision models — SCRFD for face detection and ArcFace for embedding generation — with DBSCAN clustering and FAISS vector search, the system delivers accurate, scalable face recognition and organization out of the box.

The integration of **Google Gemini** as both a multimodal event classifier and the core reasoning engine of the Agentic Chat Assistant transforms the system from a simple photo gallery into a genuinely conversational AI companion. Users can ask natural language questions about their photo collection, request deliveries, and trigger organizational workflows without touching any menus or settings.

The delivery integrations with **Brevo SMTP** and **Twilio WhatsApp**, combined with comprehensive delivery history tracking, provide a professional-grade sharing workflow. The **Cloudinary** integration ensures reliable, CDN-accelerated image hosting with automatic transformations, while the **CLIP semantic search** capability enables discovery of photos by scene, activity, or concept — not just face.

Built as a single-service Flask application with a clean SQLite schema supporting full multi-tenant isolation, the system is easy to deploy — either directly with `python app.py` or as a Docker container. The comprehensive suite of scratch test scripts and the modular API design make it straightforward to extend with new AI capabilities, cloud backends, or delivery channels.

In conclusion, Photo Management Agentic AI successfully delivers on the vision of an intelligent photo companion — one that recognizes, organizes, searches, and delivers with precision and conversational ease. It bridges the gap between advanced machine learning and everyday user needs, setting a strong foundation for next-generation AI-enhanced digital memory management.